

CSL351 : COMPUTER NETWORKS
Assignment - 10

Name - Ashish Ranjan

ID no. - 12340370

Google Drive Link :

<https://drive.google.com/drive/folders/1NI AeOc7XXshYGrpoYkgQfXvz663lt87u?usp=sharing>

The drive link has the different code files for both the Part1 and Part2 in their respective folders.

Folder Structure :

- For Part1 Folder there are 3 code files , one dns record file and one Readme file, Server.c as DNS server, videosever.cpp as UDP videosever and client.cpp as TCP-UDP client , dns_records.txt containing the domain_name to ip_addr resolving and README.txt for compilation and running commands.
- For Part2 Folder first question has two codes q1server.c and q1client.c for the rtt measurement,
Second question has three code files q2server.c , q2client.c , q2plot.py for running and generating the plot respectively which is generated via results.csv directed by the '>' command as mentioned in README.
The delayplot.png and throughputplot.png are the output of q2plot.py
Third question has one code file q3traceroute.c

Report from next page.

PART 1

- Server.c (DNS Server TCP based)

```
Server > TCP > C Server.c > main(int, char* [])
1  /* A simple server in the internet domain using TCP.  The port number is passed as an argument */
2
3  #include <stdio.h>
4  //This header file contains declarations used in most input and output and is typically included in all C programs.
5  #include <sys/types.h>
6  //This header file contains definitions of a number of data types used in system calls. These types are used in the next two include files.
7  #include <sys/socket.h>
8  //The header file socket.h includes a number of definitions of structures needed for sockets.
9  #include <netinet/in.h>
10 //The header file netinet/in.h contains constants and structures needed for internet domain addresses.
11 #include <stdlib.h>
12 #include <unistd.h>
13 #include <string.h>
14 #include <arpa/inet.h>
15
16 typedef struct {
17     int id;
18     char qname[256];
19     char ipaddr[16]; // empty in req
20 } DNS;
21
22 //This function is called when a system call fails. It displays a message about the error on stderr and then aborts the program.
23 void error(char *msg)
24 {
25     perror(msg);
26     exit(1);
27 }
28
29 //sockfd and newsockfd are file descriptors. These two variables store the values returned by the socket system call and the accept system call.
30 // portno stores the port number on which the server accepts connections.
31 // clien stores the size of the address of the client. This is needed for the accept system call.
32 // An in_addr structure within sockaddr_in structure, defined in the same header file, contains only one field, a unsigned long called s_addr. The variable serv
33
34 int main(int argc, char *argv[])
35 {
36     int sockfd, newsockfd, portno, clien;
37     char buffer[256]; // The server reads characters from the socket connection into this buffer.
38     struct sockaddr_in serv_addr, cli_addr; // A sockaddr_in is a structure containing an internet address. This structure is defined in <netinet/in.h>
39     int n; // n is the return value for the read() and write() calls; i.e. it contains the number of characters read or written.
40
41     DNS dns_req;
42     if (argc < 2) {
43         fprintf(stderr, "ERROR, no port provided\n");
44         exit(1);
45     }
46     //The user needs to pass in the port number on which the server will accept connections as an argument. This code displays an error message if the user fails to
47
48     sockfd = socket(AF_INET, SOCK_STREAM, 0); // The socket() system call creates a new socket. It takes three arguments as discussed.
49
50     if (sockfd < 0)
51         error("ERROR opening socket");
52     //The socket system call returns an entry into the file descriptor table (i.e. a small integer). This value is used for all subsequent references to this socket
53
54     bzero((char *) &serv_addr, sizeof(serv_addr)); // The function bzero() sets all values in a buffer to zero. It takes two arguments, the first is a pointer to
55
56     portno = atoi(argv[1]); // The port number on which the server will listen for connections is passed in as an argument, and this statement sets the value
57
58     serv_addr.sin_family = AF_INET; // The variable serv_addr is a structure of type struct sockaddr_in. This structure has four fields. The first field is the
59
60     serv_addr.sin_port = htons(portno); // The second field of serv_addr is unsigned short sin_port, which contain the port number. However, in network byte order
```

```
Server > TCP > C Server.c > main(int, char* [])
34 int main(int argc, char *argv[])
62     serv_addr.sin_addr.s_addr = INADDR_ANY; // The third field of sockaddr_in is a structure of type struct in_addr which contains only a single
63
64     if (bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
65         error("ERROR on binding");
66     //The bind() system call binds a socket to an address, in this case the address of the current host and port number on which the server will run.
67
68     listen(sockfd,5); // The listen system call allows the process to listen on the socket for connections. The first argument is the socket fil
69     printf("DNS server is running on port %d\n",portno);
```

```

70 while(1){
71     cli_len = sizeof(cli_addr);
72     newsockfd = accept(sockfd, (struct sockaddr *)&cli_addr, &cli_len);
73     if (newsockfd < 0)
74         error("ERROR on accept");
75     bzero((char *) &dns_req, sizeof(DNS));
76     n=read(newsockfd, &dns_req, sizeof(DNS));
77     if (n<0) error("ERROR reading from socket DNS");
78     printf("recv DNS req - ID: %d, Domain: %s\n", dns_req.id, dns_req.qname);
79
80
81
82     FILE *file = fopen("dns_records.txt", "r");
83     char file_domain[256], file_ip[16];
84     int found = 0;
85     strcpy(dns_req.ipaddr, "NOT_FOUND");
86
87     if (file) {
88         while (fscanf(file, "%s %s", file_domain, file_ip) != EOF) {
89             if (strcmp(file_domain, dns_req.qname) == 0) {
90                 strcpy(dns_req.ipaddr, file_ip);
91                 found = 1;
92                 break;
93             }
94         }
95         fclose(file);
96     } else {
97         perror("ERROR opening dns_records.txt");
98     }
99
100     n=write(newsockfd, &dns_req, sizeof(DNS));
101     if (n < 0) error("ERROR writing to socket");
102
103     close(newsockfd);
104     // The accept() system call causes the process to block until a client connects to the server. Thus, it wakes up the process when a connection from a client has
105
106
107
108     //Note that we would only get to this point after a client has successfully connected to our server. This code initializes the buffer using the bzero() function.
109     //printf("\n New Connection from client %s:%d: \n ", inet_ntoa(cli_addr.sin_addr), ntohs(cli_addr.sin_port));
110     // bzero(buffer,256);
111     // n = read(newsockfd,buffer,255);
112     // if (n < 0) error("ERROR reading from socket");
113     // printf("\n received from client port: %d ",ntohs(cli_addr.sin_port));
114     //printf("\n Here is the message: %s\n",buffer);
115
116     //Once a connection has been established, both ends can both read and write to the connection. Naturally, everything written by the client will be read by the server.
117     // n = write(newsockfd,"I got your message",18);
118     // if (n < 0) error("ERROR writing to socket");
119
120     //This terminates main and thus the program. Since main was declared to be of type int as specified by the ascii standard, many compilers complain if it does not
121     return 0;
122 }
123

```

Explanation :

I have used the sample codes provided for the TCP server and made necessary changes needed to make this a DNS Server. This reads a file name dns_records.txt and searched the given domain name and returns the corresponding IP address. I initialize the server address using a sockaddr_in structure, setting INADDR_ANY so the server binds to all available network interfaces, and pushing the command line port number through htons() to ensure it's in the correct network byte order. One specific hardcoded detail here is the listen(sockfd, 5) call; I set the connection backlog queue strictly to 5. This means if more than five clients try to connect simultaneously while the server is busy processing, the operating system will drop the excess connection attempts. Once the server is running, it hits an infinite while(1) loop where it blocks at accept(). When a connection comes in, the OS gets a new file descriptor dedicated entirely to

that client session. To handle the data, I first zeroed DNS struct using `bzero()` and then pull the incoming payload directly into the struct's memory space using `read(..., &dns_req, sizeof(DNS))` assuming that both the client and server are compiled with the exact same memory alignment, struct padding. The domain resolution I implemented a file lookup. For every single incoming request, the server dynamically opens a file named `dns_records.txt` in read mode. It uses a `while (fscanf(...) != EOF)` loop to parse the text line-by-line comparing the requested qname against each domain entry using `strcmp()`. If it hits an exact match, it copies the corresponding IP address into the struct's `ipaddr` field and breaks the loop. If it reads through the entire file without finding a match, it falls back to a "NOT_FOUND" string. the server writes that modified struct right back down the TCP to the client and immediately closes the client socket.

- **videoserver.cpp (UDP based)**

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<sys/types.h>
4 #include<sys/socket.h>
5 #include<netinet/in.h>
6 #include<unistd.h>
7 #include <arpa/inet.h>
8 #include <opencv2/opencv.hpp>
9
10 #define MAXLINE 65535
11 #define PORT 8000
12 void error(const char *msg) {
13     perror(msg);
14     exit(1);
15 }
16
17 int main(){
18
19     int socketDescriptor = socket(AF_INET, SOCK_DGRAM, 0);
20     if (socketDescriptor < 0) error("ERROR opening socket");
21
22     struct sockaddr_in serverAddress, clientAddress;
23     socklen_t addressLength;
24     char message[MAXLINE];
25
26     bzero(&serverAddress, sizeof(serverAddress));
27     serverAddress.sin_family = AF_INET;
28     serverAddress.sin_addr.s_addr=INADDR_ANY;
29     serverAddress.sin_port=htons(PORT);
30
31     if (bind(socketDescriptor, (struct sockaddr*)&serverAddress, sizeof(serverAddress)) < 0)
32         error("ERROR on binding");
33
34     struct timeval tv;
35     tv.tv_sec = 0;
36     tv.tv_usec = 200000;
37     setsockopt(socketDescriptor, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
38
39     cv::VideoCapture capture(0);
40     if (!capture.isOpened()) {
41         error("ERROR opening webcam");
42     }
43
44     printf("video server started on port %d\n", PORT);
45
46     std::vector<int> encode_params = {cv::IMWRITE_JPEG_QUALITY, 40};
47
48     while(1){
49
50         addressLength = sizeof(clientAddress);
51         bzero(message, MAXLINE);
52
53         int n= recvfrom(socketDescriptor,message,MAXLINE,0,(struct sockaddr*)&clientAddress,&addressLength);
```

```

54
55 if (n<0){
56     continue;
57 }
58
59 cv::Mat frame;
60 capture >> frame;
61 if (frame.empty()) continue;
62
63 std::vector<uchar> encoded_img;
64 cv::imencode(".jpg", frame, encoded_img, encode_params);
65
66 if (!encoded_img.empty()) {
67     sendto(socketDescriptor, encoded_img.data(), encoded_img.size(), 0, (struct sockaddr*)&clientAddress, addressLength);
68 }
69 }
70
71 capture.release();
72 return 0;
73 }

```

Explanation :

I have written this in c++ as the opencv was causing some issues during compilation so i have used the latest opencv version supported by c++ to achieve this. This code is based on the UDP server code given in the sample with necessary modification.

The networking setup begins with socket creation using `socket(AF_INET, SOCK_DGRAM, 0)`, which establishes an IPv4 UDP socket. Next, i set up a `sockaddr_in` structure and binds the socket to `INADDR_ANY` on the input argument port and if not given then on 8790, instructing the operating system to listen for incoming UDP packets on all available network interfaces. the timeout configuration as specified by the question is handled via the `setsockopt` function, which establishes a `SO_RCVTIMEO` (receive timeout) 0.2 seconds. this timeout is necessary to prevent the server from freezing if the client suddenly stops sending requests. For the camera and OpenCV setup, `cv::VideoCapture capture(0)` is used to hook the default webcam on my system. The compression parameters are defined by the `encode_params` vector, which sets the `IMWRITE_JPEG_QUALITY` to 40. This deliberately low-quality setting is used to heavily compress the image. A raw, uncompressed webcam frame is massive, this compression is vital to avoid complex packet fragmentation and ensure the entire frame can typically fit into a single UDP datagram. The core of the program that is a `while(1)` acts as an infinite loop to serve the video. Inside this loop, the server waits for a ping by calling `recvfrom`, sitting idle until a client sends a message. the server only transmits a frame after being pinged, rather than broadcasting. If 0.2 seconds elapse without a ping, `recvfrom` returns a value less than 0, which is caught by an `if (n < 0)`. Once a ping is

successfully received, capture >> frame grabs the latest image from the webcam, and cv::imencode(".jpg", ...) converts the raw OpenCV matrix into a compressed array of bytes. Finally, the sendto function transmits this compressed byte array back to the exact IP address and port of the client that sent the initial ping.

- **Client.cpp (Both TCP and UDP)**

```
Client > client.cpp > MAXLINE
1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/socket.h>
4  #include <netinet/in.h>
5  #include <netdb.h>
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <unistd.h>
9  #include <string.h>
10 #include <arpa/inet.h>
11 #include <time.h>
12 #include <opencv2/opencv.hpp>
13
14 #define MAXLINE 65535
15 #define VIDEO_PORT 8000
16 typedef struct {
17     int id;
18     char qname[256];
19     char ipaddr[16];
20 } DNS;
21
22
23 void error(char *msg)
24 {
25     perror(msg);
26     exit(0);
27 }
28
29 //The error() function is identical to that in the server, as are the variables sockfd, portno, and n. The variable serv_addr will contain the ad
30
31 int main(int argc, char *argv[])
32 {
33     srand(time(NULL));
34
35     if (argc < 4) {
36         fprintf(stderr, "Usage: %s <dns_server_ip> <dns_server_port> <domain_name>\n", argv[0]);
37         exit(0);
38     }
39
40
41     int tcp_sockfd, dns_port;
42     struct sockaddr_in dns_addr;
43     struct hostent *server;
44     DNS dns_req;
45
46     dns_port=atoi(argv[2]);
47     tcp_sockfd=socket(AF_INET, SOCK_STREAM, 0);
48     if (tcp_sockfd < 0) error("ERROR opening TCP socket");
49
50     server = gethostbyname(argv[1]);
51     if (server == NULL) {
52         fprintf(stderr, "ERROR, no such host\n");
53         exit(0);
54     }
55
56     bzero((char *) &dns_addr, sizeof(dns_addr));
57     dns_addr.sin_family = AF_INET;
58     bcopy((char *)server->h_addr, (char *)&dns_addr.sin_addr.s_addr, server->h_length);
59     dns_addr.sin_port = htons(dns_port);
```

```

60
61     if (connect(tcp_sockfd, (struct sockaddr *)&dns_addr, sizeof(dns_addr)) < 0)
62         error("ERROR connecting to DNS server");
63
64
65     bzero(&dns_req, sizeof(DNS));
66     dns_req.id=rand()%65536;
67     // dns_req.transaction_id = 12340370;
68     strncpy(dns_req.qname, argv[3], 255);
69
70
71     write(tcp_sockfd, &dns_req, sizeof(DNS));
72     read(tcp_sockfd, &dns_req, sizeof(DNS));
73     close(tcp_sockfd);
74
75     printf("\nDNS Resolution Output\n");
76     printf("Domain: %s\n", dns_req.qname);
77     printf("IP Address: %s\n", dns_req.ipaddr);
78     printf("-----\n\n");
79
80     if (strcmp(dns_req.ipaddr, "NOT_FOUND") == 0) {
81         fprintf(stderr, "DNS Resolution Failed. Exiting.\n");
82         exit(1);

```

```

85     int udp_sockfd = socket(AF_INET, SOCK_DGRAM, 0);
86     struct sockaddr_in video_serv_addr;
87     char recvMessage[MAXLINE];
88
89     bzero(&video_serv_addr, sizeof(video_serv_addr));
90     video_serv_addr.sin_family = AF_INET;
91     video_serv_addr.sin_port = htons(VIDEO_PORT);
92     inet_pton(AF_INET, dns_req.ipaddr, &video_serv_addr.sin_addr);
93
94     socklen_t addressLength = sizeof(video_serv_addr);
95     cv::namedWindow("video stream", cv::WINDOW_AUTOSIZE);
96
97     printf("starting video stream. 'x' to exit.\n");
98
99     while(1) {
100         char keep_alive[] = "ALIVE";
101         sendto(udp_sockfd, keep_alive, strlen(keep_alive), 0, (struct sockaddr *)&video_serv_addr, addressLength);
102
103
104         int n=recvfrom(udp_sockfd, recvMessage, MAXLINE, 0, NULL, NULL);
105
106         if (n>0) {
107             std::vector<uchar> raw_data(recvMessage, recvMessage + n);
108             cv::Mat decoded_frame = cv::imdecode(raw_data, cv::IMREAD_COLOR);
109             if (!decoded_frame.empty()) {
110                 cv::imshow("video stream", decoded_frame);
111             }
112             char c = (char)cv::waitKey(1);
113             if (c=='x') break;
114         }
115     }
116
117     cv::destroyWindow("video stream");
118     close(udp_sockfd);
119     return 0;
120 }

```

Explanation :

This code serves as the client side part to the video server. It performs two main tasks. First it asks the DNS server to find the IP address of a target domain using a TCP connection. Then it switches to a UDP connection to request and display the live video stream from that newly discovered IP address. This code is based on the TCP client code given in the sample with necessary modification. I first verify the command line arguments to ensure it has the DNS server IP which is the loopback address to check the DNS port on this loopback address , and the target domain name. I then create a standard TCP socket using `socket(AF_INET, SOCK_STREAM, 0)`. TCP is used here because it stated in the question to use TCP for DNS communication. It connects to the DNS server and populates a custom

DNS data structure. This structure holds a randomly generated transaction ID and the requested domain name. The client transmits this structure over the network using write. It then pauses to read the response back into the same structure. The custom DNS server fills the ipaddr field with the real IP address. If the returned IP is "NOT_FOUND", i have ensured that the program terminates to prevent crashing in the next step. I then creates a brand new UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`. It takes the IP address received from the DNS server and converts it into a binary network format using the `inet_pton` function. It attaches this IP and the input argument port to a `video_serv_addr` structure. Before starting the network loop it also initializes a graphical window using `cv::namedWindow` to create a blank canvas for the incoming video frames. The client then enters a `while(1)` loop to fetch and display the continuous video feed. It first sends a brief "ALIVE" text string to the video server using `sendto`. This acts as a trigger ping. The client then waits to catch the incoming compressed frame using `recvfrom`. When the raw byte data arrives it gets copied into a standard C++ vector. The `cv::imdecode` function decompresses that raw JPEG byte array back into a usable OpenCV image matrix. `cv::imshow` takes that matrix and renders it onto the screen. Finally a `cv::waitKey(1)` command gives the graphical interface one millisecond to update its display. i have then used the 'x' key to close the program.

Output :

- **Server.c (DNS Server TCP based)**

COMMAND USED :

```
gcc Server.c -o server
```

```
./server 8080
```

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_10/Server/TCP$ gcc Server.c -o server
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_10/Server/TCP$ ./server 8080
DNS server is running on port 8080
```

- **videoserver.cpp (UDP based)**

COMMAND USED :

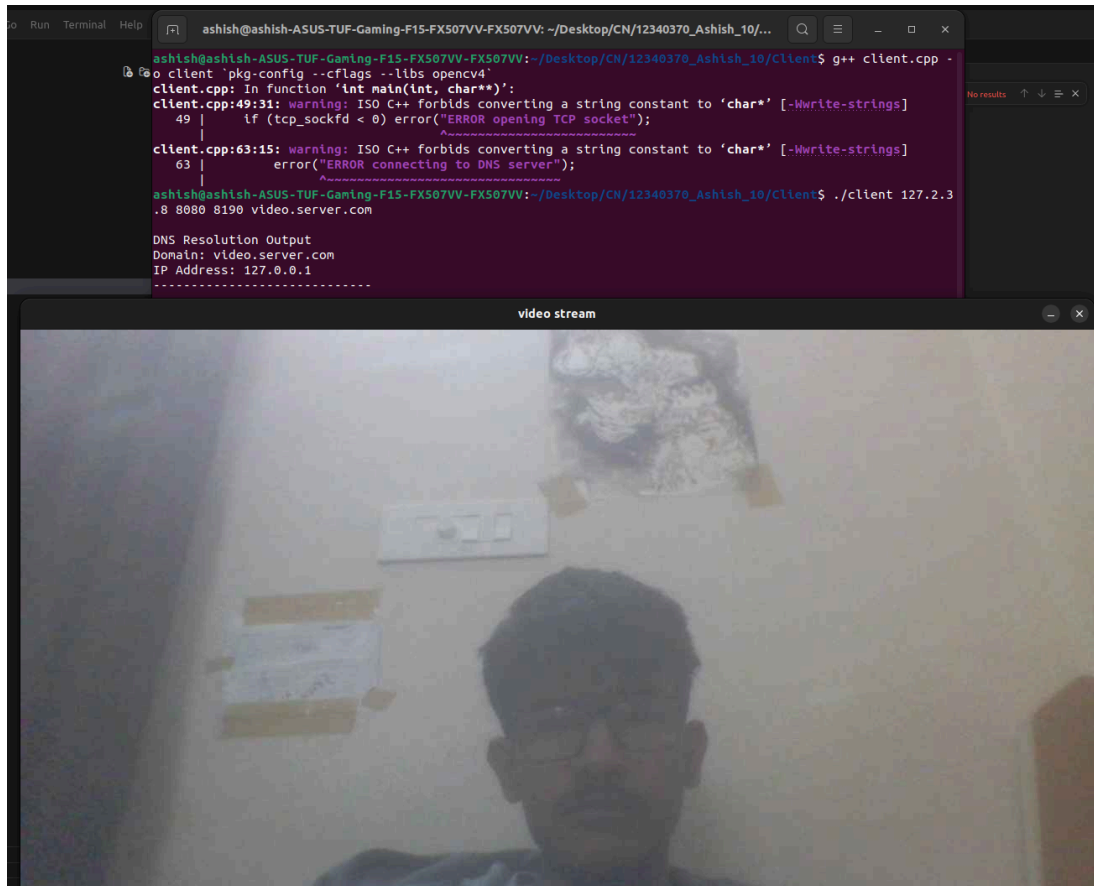
```
g++ videoserver.cpp -o videoserver `pkg-config --cflags --libs opencv4`  
./videoserver 8190
```

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Server/UDP$ g++ videoserver.cpp -o videoserver `pkg-config --cflags --libs opencv4`  
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Server/UDP$ ./videoserver 8190  
[ WARN:0] global ./modules/videoio/src/cap_gstreamer.cpp (1100) open OpenCV | GStreamer warning: Cannot query video position: status=0, value=-1, duration=-1  
video server started on port 8190
```

- **Client.cpp (Both TCP and UDP based)**

COMMAND USED :

```
g++ client.cpp -o client `pkg-config --cflags --libs opencv4`  
./client 127.0.0.1 8080 8190 video.server.com
```



PART 2

Q1.) Measuring RTT

server.c

```
server.c > main(int, char * [])
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

void error(const char *msg) {
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(1);
    }

    int sockfd;
    struct sockaddr_in serv_addr, cli_addr;
    socklen_t clilen;
    int portno = atoi(argv[1]);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) error("ERROR opening socket");

    bzero((char *) &serv_addr, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = INADDR_ANY;
    serv_addr.sin_port = htons(portno);

    if (bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0) {
        error("ERROR on binding");
    }

    printf("UDP Echo Server is running on port %d\n", portno);

    char *buffer = (char *)malloc(65535);
    if (buffer == NULL) error("ERROR allocating memory");

    while (1) {
        clilen = sizeof(cli_addr);

        int n = recvfrom(sockfd, buffer, 65535, 0, (struct sockaddr *) &cli_addr, &clilen);
        if (n < 0) {
            perror("ERROR in recvfrom");
            continue;
        }

        int sent = sendto(sockfd, buffer, n, 0, (struct sockaddr *) &cli_addr, clilen);
        if (sent < 0) {
            perror("ERROR in sendto");
        }
    }

    free(buffer);
    close(sockfd);
}
```

Explanation :

I took the port number from `argv[1]` and initialized my UDP socket with `socket(AF_INET, SOCK_DGRAM, 0)`. I zeroed out my `serv_addr` struct with `bzero()`, set the family to `AF_INET`, the IP to `INADDR_ANY` so it listens on all interfaces, and the port to `htons(portno)`. I linked the socket to the port using the `bind()` function. Because UDP packets can theoretically be up to 64KB, I used `malloc(65535)` to create a massive generic buffer. The core part of the server is an infinite while (1) loop. Inside this loop, I called `recvfrom(sockfd, buffer, 65535, 0, (struct sockaddr *) &cli_addr, &clilen)`. This function blocks until a client sends something. The moment it catches a packet, it populates the buffer and the `cli_addr` struct with the sender's info, and returns the byte length into the variable `n`. I immediately took that exact same buffer, length `n`, and `cli_addr`, and passed them directly into `sendto()` to the payload right back to the client.

client.c

```
client.c:1:1: warning: 'char *[]' is deprecated [-Wdeprecated-declarations]
1  int main(int argc, char *[])
    ^
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/types.h>
6  #include <sys/socket.h>
7  #include <netinet/in.h>
8  #include <arpa/inet.h>
9  #include <sys/time.h>
10 #include <netdb.h>
11
12 typedef struct {
13     int seq_num;
14     struct timeval send_time;
15 } packethead;
16
17 void error(const char *msg) {
18     perror(msg);
19     exit(1);
20 }
21
22 int main(int argc, char *argv[]) {
23     if (argc < 6) {
24         fprintf(stderr, "Usage: %s <serverip> <port> <msgcount> <intervalms> <packetsize>\n", argv[0]);
25         exit(1);
26     }
27     char *serverip=argv[1];
28     int portno=atoi(argv[2]);
29     int msgcount=atoi(argv[3]);
30     int intervalms=atoi(argv[4]);
31     int packetsz=atoi(argv[5]);
32     if (packetsz < sizeof(packethead)) {
33         fprintf(stderr, "Packet size must be at least %lu bytes\n", sizeof(packethead));
34         exit(1);
35     }
36
37     int sockfd;
38     struct sockaddr_in serv_addr;
```

```

int sockfd;
struct sockaddr_in serv_addr;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);
if (sockfd < 0) error("ERROR opening socket");

bzero((char *) &serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(portno);
if (inet_pton(AF_INET, serverip, &serv_addr.sin_addr) <= 0) {
    error("ERROR invalid server IP address");
}

struct timeval timeout;
timeout.tv_sec = 1;
timeout.tv_usec = 0;
if (setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0) {
    error("ERROR setting socket timeout");
}

char *buffer = (char *)malloc(packetesz);
if (buffer == NULL) error("ERROR allocating memory");
memset(buffer, 'A', packetesz);

printf("PING %s:%d %d bytes of data.\n", serverip, portno, packetesz);

int rcvcnt = 0;
int lost_count = 0;
double trtt = 0.0;

for (int i=1;i<=msgcount;i++){
    packethead *head=(packethead *)buffer;
    head->seq_num = i;
    gettimeofday(&head->send_time, NULL);

    int n=sendto(sockfd,buffer,packetesz,0,(struct sockaddr *)&serv_addr,sizeof(serv_addr));
    if (n<0){
        perror("ERROR writing to socket");
        lost_count++;
        continue;
    }

    struct sockaddr_in from_addr;
    socklen_t from_len=sizeof(from_addr);
    n=recvfrom(sockfd,buffer,packetesz,0,(struct sockaddr *)&from_addr,&from_len);

    if (n < 0) {
        printf("Request timeout for icmp_seq %d\n", i);
        lost_count++;
    } else {
        struct timeval rcvtime;
        gettimeofday(&rcvtime, NULL);

        packethead *rcvhead = (packethead *)buffer;

```

```

        if (rcvhead->seq_num == i) {
            double rtt = (rcvtime.tv_sec - rcvhead->send_time.tv_sec) * 1000.0 + (rcvtime.tv_usec - rcvhead->send_time.tv_usec) / 1000.0;
            printf("%d bytes from %s: seq=%d time=%.3f ms\n", n, serverip, rcvhead->seq_num, rtt);

            rcvcnt++;
            trtt += rtt;
        } else {
            printf("Received out-of-order packet. Expected seq=%d, got seq=%d\n", i, rcvhead->seq_num);
        }
    }

    usleep(intervalms * 1000);
}

printf("\n--- %s ping statistics ---\n", serverip);
double loss=((double)lost_count / msgcount) * 100.0;

```

```

printf("----- ping statistics -----\n", serverip);
double loss = ((double)lost_count / msgcount) * 100.0;
printf("%d packets transmitted, %d received, %.1f%% packet loss\n", msgcount, rcvcount, loss);

if (rcvcount > 0) {
    printf("Average RTT: %.3f ms\n", trtt / rcvcount);
}

free(buffer);
close(sockfd);
return 0;
}

```

Explanation :

I started by taking the command-line arguments and converting them using `atoi()` into my core variables: `serverip`, `portno`, `msgcount`, `intervalms`, and `packetsz`. To track the timestamps, I defined a custom `packethead` struct containing an `int seq_num` and a `struct timeval send_time`. I created the socket using `sockfd = socket(AF_INET, SOCK_DGRAM, 0)` and used `inet_pton()` to convert the server IP string into the network address structure `serv_addr.sin_addr`. To prevent the client from hanging if a packet was dropped, I used the `setsockopt()` function with the `SO_RCVTIMEO` flag, passing in a timeout struct set to exactly 1 second (`tv_sec = 1`, `tv_usec = 0`). I then used `malloc()` to allocate a buffer of `packetsz` bytes and filled it with 'A's using `memset()`. Inside my main for loop (running `msgcount` times), I cast the start of my buffer to a `packethead` pointer, assigned the current sequence number to `head->seq_num`, and grabbed the exact time using `gettimeofday(&head->send_time, NULL)`. I send the packet over the network using `sendto()`. When listening for the reply, I called `recvfrom()` using a `from_addr` struct. If it returned a value greater than 0 showing no error encountered I called `gettimeofday(&rcvtime, NULL)` again. By casting the received buffer back to a `packethead`, I could verify `rcvhead->seq_num == i` and calculate the round-trip time (RTT) by subtracting `rcvhead->send_time` from `rcvtime`. I kept an overall count of lost packets in the `lost_count` variable and total time in `trtt` to print the final statistics.

Output :

server.c

COMMAND USED :

```
gcc q1server.c -o q1server
./q1server 9000
```

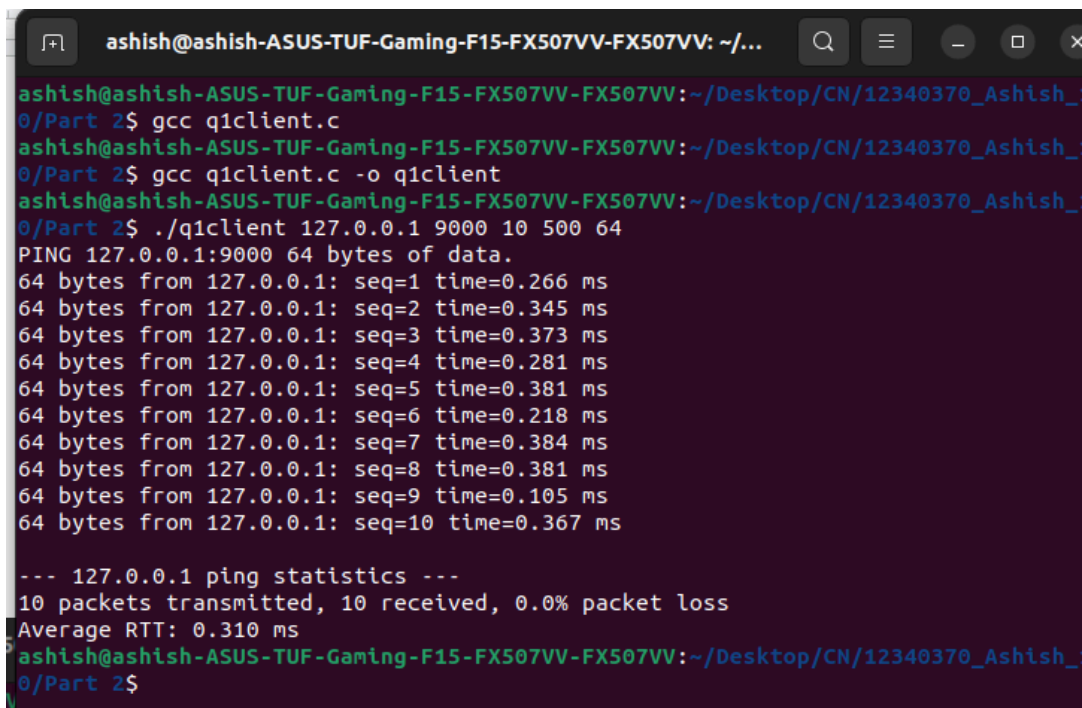
A terminal window with a dark background and light green text. The window title is 'ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/...'. The terminal shows the following commands and output:

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$ gcc q1server.c
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$ gcc q1server.c -o q1server
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$ ./q1server 9000
UDP Echo Server is running on port 9000
█
```

client.c

COMMAND USED :

```
gcc q1client.c -o q1client
./q1client 127.0.0.1 9000 10 500 64
```

A terminal window with a dark background and light green text. The window title is 'ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/...'. The terminal shows the following commands and output:

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$ gcc q1client.c
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$ gcc q1client.c -o q1client
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$ ./q1client 127.0.0.1 9000 10 500 64
PING 127.0.0.1:9000 64 bytes of data.
64 bytes from 127.0.0.1: seq=1 time=0.266 ms
64 bytes from 127.0.0.1: seq=2 time=0.345 ms
64 bytes from 127.0.0.1: seq=3 time=0.373 ms
64 bytes from 127.0.0.1: seq=4 time=0.281 ms
64 bytes from 127.0.0.1: seq=5 time=0.381 ms
64 bytes from 127.0.0.1: seq=6 time=0.218 ms
64 bytes from 127.0.0.1: seq=7 time=0.384 ms
64 bytes from 127.0.0.1: seq=8 time=0.381 ms
64 bytes from 127.0.0.1: seq=9 time=0.105 ms
64 bytes from 127.0.0.1: seq=10 time=0.367 ms

--- 127.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0.0% packet loss
Average RTT: 0.310 ms
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_1
0/Part 2$
```

Q2.) Measuring throughput with iperf like application

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

void error(const char *msg) {
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(1);
    }

    int sockfd;
    struct sockaddr_in serv_addr, cli_addr;
    socklen_t clilen;
    int portno = atoi(argv[1]);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) error("ERROR opening socket");

    bzero((char *) &serv_addr, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = INADDR_ANY;
    serv_addr.sin_port = htons(portno);

    if (bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
        error("ERROR on binding");

    printf("iPerf-like udp Server running on port %d\n", portno);

    char *buffer = (char *)malloc(65535);
    while (1) {
        clilen = sizeof(cli_addr);
        int n = recvfrom(sockfd, buffer, 65535, 0, (struct sockaddr *) &cli_addr, &clilen);
        if (n > 0) {
            sendto(sockfd, buffer, n, 0, (struct sockaddr *) &cli_addr, clilen);
        }
    }

    free(buffer);
    close(sockfd);
    return 0;
}
```

Explanation :

This file is exactly like that of q1server.c. I set up the sockfd, serv_addr, and called bind() exactly as before. I allocated the 65KB buffer using malloc(65535). Inside this loop, I strictly call int n = recvfrom(...) and immediately check if (n > 0). If valid data arrived, I call sent = call sendto(...) and check again if its negative.

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/time.h>
#include <pthread.h>

#define MAXI 3600

typedef struct {
    int seq_num;
    struct timeval send_time;
} packethead;

typedef struct {
    long bytesrcv;
    double trtt;
    int packetsrcv;
} secondsts;

secondsts stats[MAXI];
int running = 1;
int sockfd;
struct timeval start_test_time;

void error(const char *msg) {
    perror(msg);
    exit(1);
}

void *receive_packets(void *arg) {
    char buffer[65535];
    struct sockaddr_in from_addr;
    socklen_t from_len = sizeof(from_addr);

    while (running) {
        int n = recvfrom(sockfd, buffer, 65535, 0, (struct sockaddr *) &from_addr, &from_len);
        if (n > 0) {
            struct timeval rcvtime;
            gettimeofday(&rcvtime, NULL);

            packethead *hdr = (packethead *)buffer;
            double rtt = (rcvtime.tv_sec - hdr->send_time.tv_sec) * 1000.0 + (rcvtime.tv_usec - hdr->send_time.tv_usec) / 1000.0;

            int secindex = rcvtime.tv_sec - start_test_time.tv_sec;
            if (secindex >= 0 && secindex < MAXI) {
                stats[secindex].bytesrcv += n;
                stats[secindex].trtt += rtt;
                stats[secindex].packetsrcv++;
            }
        }
    }
    return NULL;
}
```

```

void *receive_packets(void *arg) {

int main(int argc, char *argv[]) {
    if (argc < 6) {
        fprintf(stderr, "Usage: %s <server_ip> <port_no> <duration_sec> <interval_us> <packet_size>\n", argv[0]);
        exit(1);
    }

    char *serverip = argv[1];
    int portno = atoi(argv[2]);
    int duration = atoi(argv[3]);
    int intervalus = atoi(argv[4]);
    int packetsz = atoi(argv[5]);

    if (duration > MAXI) duration = MAXI;
    memset(stats, 0, sizeof(stats));

    struct sockaddr_in serv_addr;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) error("ERROR opening socket");

    struct timeval timeout = {0, 100000};
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout));

    bzero((char *) &serv_addr, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(portno);
    inet_pton(AF_INET, serverip, &serv_addr.sin_addr);

    char *buffer = (char *)malloc(packetsz);
    memset(buffer, 'A', packetsz);

    gettimeofday(&start_test_time, NULL);
    pthread_t rcvthread;
    if (pthread_create(&rcvthread, NULL, receive_packets, NULL) != 0) {
        error("Failed to create receiver thread");
    }

    int seq = 1;
    struct timeval current_time;
    gettimeofday(&current_time, NULL);

    fprintf(stderr, "testing for %d seconds. sending packets\n", duration);

    while ((current_time.tv_sec - start_test_time.tv_sec) < duration) {
        packethead *header = (packethead *)buffer;
        header->seq_num = seq++;
        gettimeofday(&header->send_time, NULL);

        sendto(sockfd, buffer, packetsz, 0, (struct sockaddr *) &serv_addr, sizeof(serv_addr));

        if (intervalus > 0) {
            usleep(intervalus);
        }
        gettimeofday(&current_time, NULL);
    }

    running = 0;
    pthread_join(rcvthread, NULL);

```

```

printf("Second,Throughput(Mbps),AvgRTT(ms)\n");
for (int i = 0; i < duration; i++) {
    double mbps=(stats[i].bytesrcv*8.0)/1000000.0;
    double avgsrtt = 0.0;
    if (stats[i].packetsrcv>0){
        avgsrtt=stats[i].trtt/stats[i].packetsrcv;
    }
    printf("%d,%.3f,%.3f\n", i + 1, mbps, avgsrtt);
}

free(buffer);
close(sockfd);
return 0;
}

```

Explanation :

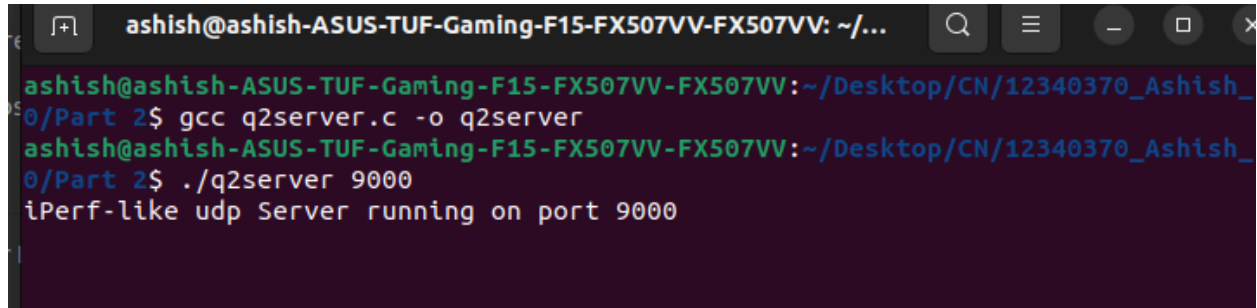
I have used multithreading under the `pthread.h` library in this part of the question. I defined a `secondsts` struct to hold `bytesrcv`, `trtt`, and `packetsrcv`, and created a global array `stats[MAXI]` so I could keep my metrics for every second. In the `main()` function, after setting up the `sockfd` and `serv_addr` like we did in the ping tool, I took a baseline timestamp into `start_test_time` using `gettimeofday()`. I then start a background thread using `pthread_create(&rcvthread, NULL, receive_packets, NULL)`. The main thread's job was strictly to send packets continuously to the server. I ran a while loop checking if $(\text{current_time.tv_sec} - \text{start_test_time.tv_sec}) < \text{duration}$. Inside this loop, I populated the packethead with an incrementing `seq` and `send_time`, fired it off with `sendto()`, and then paused briefly using `usleep(intervalus)` to control the bandwidth rate. Meanwhile, my `receive_packets` thread was running an infinite while (running) loop, constantly calling `recvfrom()`. When a packet arrived, it extracted the `rcvtime`, calculated the `rtt`, and then figured out where it belonged to by calculating `int secindex = rcvtime.tv_sec - start_test_time.tv_sec`. It then safely updated the `stats[secindex]` variables. Once the loop hits the duration, I set `running = 0`, waited for the receiver thread to finish with `pthread_join(rcvthread, NULL)`, and then looped through the `stats` array to calculate and print the `mbps` (megabits per second) and `avgsrttp`.

Output :

server

COMMAND USED :

```
gcc q2server.c -o q2server
./q2server 9000
```

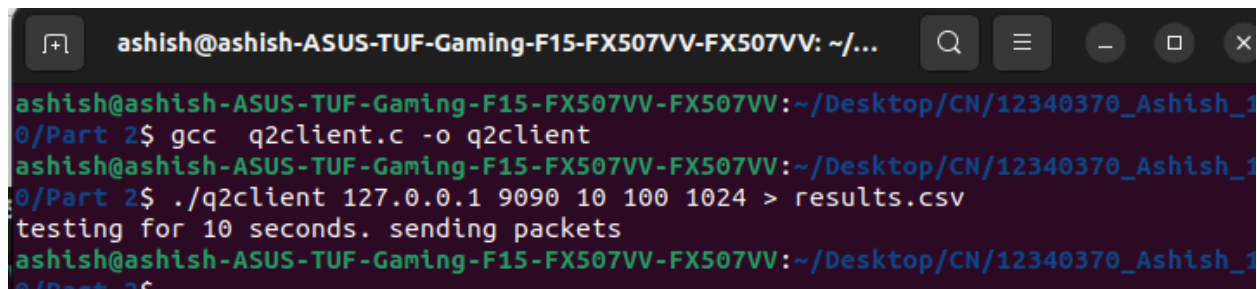
A terminal window with a dark background and light green text. The window title is 'ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/...'. The user enters 'gcc q2server.c -o q2server' and then './q2server 9000'. The output is 'iPerf-like udp Server running on port 9000'.

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/...
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Part 2$ gcc q2server.c -o q2server
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Part 2$ ./q2server 9000
iPerf-like udp Server running on port 9000
```

client

COMMAND USED :

```
gcc q2client.c -o q2client
./q2client 127.0.0.1 9090 10 100 1024 > results.csv
```

A terminal window with a dark background and light green text. The window title is 'ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/...'. The user enters 'gcc q2client.c -o q2client' and then './q2client 127.0.0.1 9090 10 100 1024 > results.csv'. The output is 'testing for 10 seconds. sending packets'.

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/...
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Part 2$ gcc q2client.c -o q2client
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Part 2$ ./q2client 127.0.0.1 9090 10 100 1024 > results.csv
testing for 10 seconds. sending packets
```

Output of results.csv :

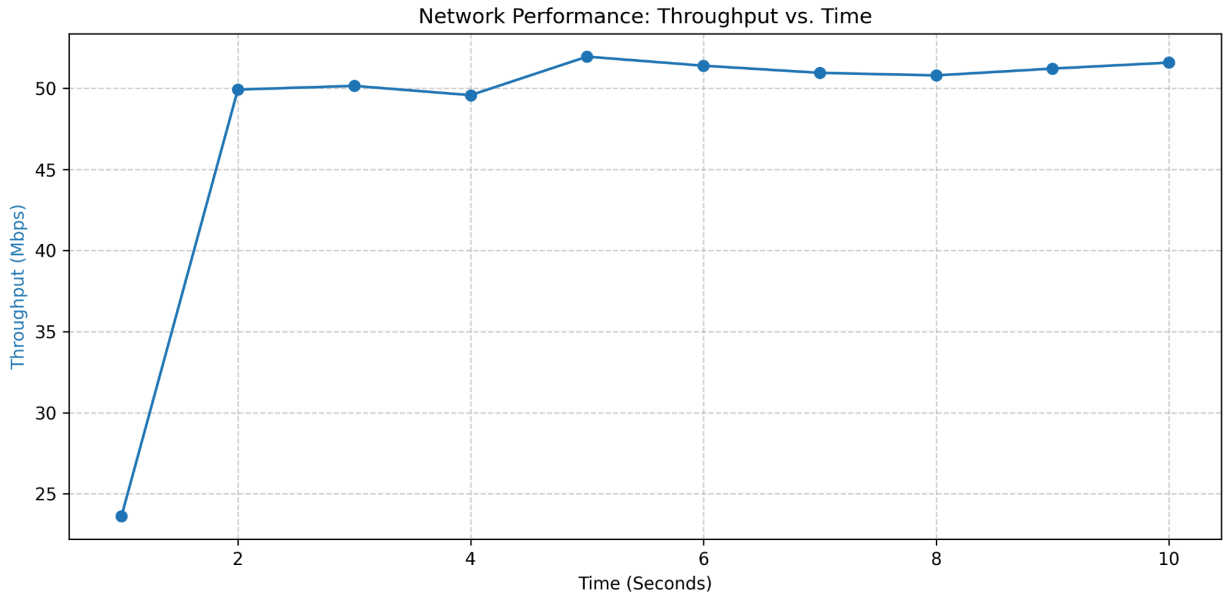
A terminal window with a dark background and light green text. The window title is 'ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Part 2'. The user enters 'cat results.csv'. The output is a table with 10 rows of data.

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV: ~/Desktop/CN/12340370_Ashish_10/Part 2$ cat results.csv
Second,Throughput(Mbps),AvgRTT(ms)
1,23.618,0.041
2,49.914,0.038
3,50.143,0.034
4,49.570,0.036
5,51.945,0.022
6,51.388,0.024
7,50.946,0.026
8,50.790,0.029
9,51.208,0.026
10,51.577,0.022
```

Plots :

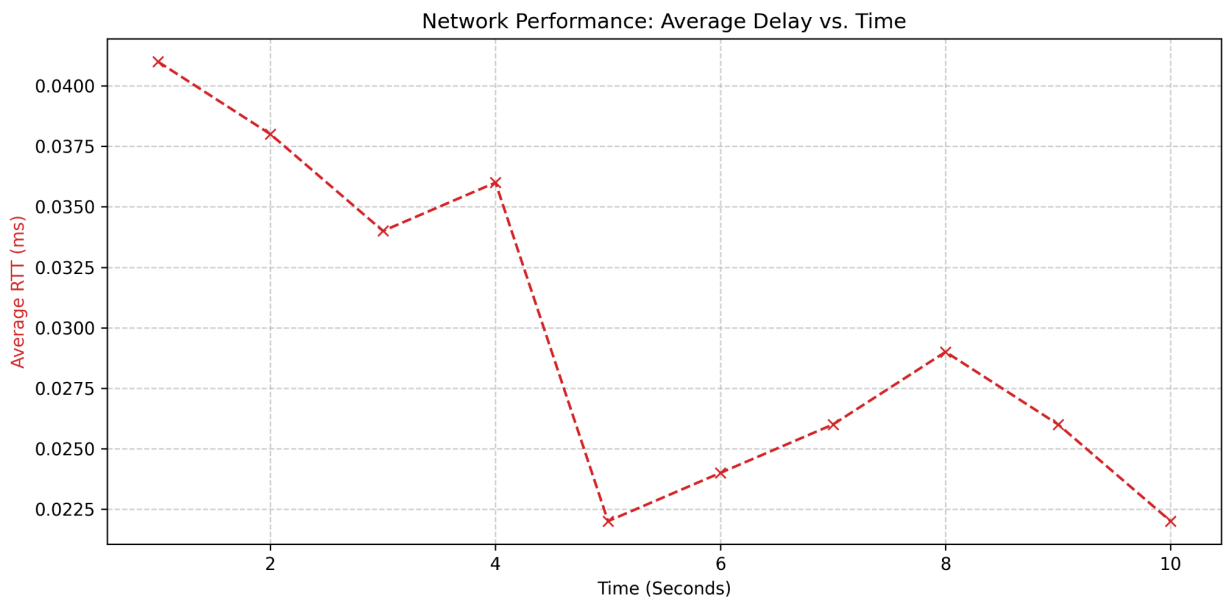
1.) Throughput vs times

COMMAND USED : python3 q2plot.py



2.) Average delay vs time

COMMAND USED : python3 q2plot.py



Q3.) traceroute

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <sys/time.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <netinet/udp.h>

#define MAXHOPS 30
#define PROBESPERHOP 3
#define RECVTIMEOUT 1
#define DESTPORT 33434
#define PACKETSZ 52

void error(const char *msg) {
    perror(msg);
    exit(1);
}

double getrtt(struct timeval *start, struct timeval *end) {
    return (end->tv_sec - start->tv_sec) * 1000.0 +
        (end->tv_usec - start->tv_usec) / 1000.0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: %s <destination_ip_or_domain>\n", argv[0]);
        fprintf(stderr, "run with sudo!\n");
        exit(1);
    }

    const char *targhost = argv[1];
    struct hostent *host_info = gethostbyname(targhost);
    if (!host_info) {
        fprintf(stderr, "ERROR: Could not resolve hostname %s\n", targhost);
        exit(1);
    }

    struct sockaddr_in destaddr;
    memset(&destaddr, 0, sizeof(destaddr));
    destaddr.sin_family = AF_INET;
    memcpy(&destaddr.sin_addr, host_info->h_addr, host_info->h_length);
    destaddr.sin_port = htons(DESTPORT);

    char dest_ip_str[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &destaddr.sin_addr, dest_ip_str, sizeof(dest_ip_str));

    printf("traceroute to %s (%s), %d hops max, %d byte packets\n",
        targhost, dest_ip_str, MAXHOPS, PACKETSZ);

    int sendsock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sendsock < 0) error("ERROR creating UDP socket");

    int recv_sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
```

```

int main(int argc, char *argv[]) {
    if (recv_sock < 0) {
        error("ERROR creating Raw socket , used sudo ?");
    }

    struct timeval timeout;
    timeout.tv_sec = RECVMSECS;
    timeout.tv_usec = 0;
    if (setsockopt(recv_sock, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0) {
        error("ERROR setting receive timeout");
    }

    char sendbuf[PACKETSZ];
    memset(sendbuf, 'A', sizeof(sendbuf));
    int destreached = 0;

    for (int ttl = 1; ttl <= MAXHOPS && !destreached; ttl++) {

        if (setsockopt(sendsock, IPPROTO_IP, IP_TTL, &ttl, sizeof(ttl)) < 0) {
            error("ERROR setting TTL");
        }

        printf("%2d ", ttl);

        char lastrouterip[INET_ADDRSTRLEN] = "";

        for (int probe = 0; probe < PROBESPERHOP; probe++) {

            destaddr.sin_port = htons(DESTPORT + ttl + probe);

            struct timeval start_time, end_time;
            gettimeofday(&start_time, NULL);

            if (sendto(sendsock, sendbuf, sizeof(sendbuf), 0,
                (struct sockaddr*)&destaddr, sizeof(destaddr)) < 0) {
                error("ERROR sending probe");
            }

            char recv_buf[512];
            struct sockaddr_in router_addr;
            socklen_t addr_len = sizeof(router_addr);

            int recv_len = recvfrom(recv_sock, recv_buf, sizeof(recv_buf), 0,
                (struct sockaddr*)&router_addr, &addr_len);

            gettimeofday(&end_time, NULL);

            if (recv_len < 0) {
                printf(" ");
            } else {
                double rtt = getrtt(&start_time, &end_time);
                struct ip *ip_hdr = (struct ip *)recv_buf;
                int ip_hdr_len = ip_hdr->ip_hl << 2;
                struct icmp *icmp_hdr = (struct icmp *) (recv_buf + ip_hdr_len);

                char routerip[INET_ADDRSTRLEN];
                inet_ntop(AF_INET, &router_addr.sin_addr, routerip, sizeof(routerip));

```

```

                if (strcmp(lastrouterip, routerip) != 0) {
                    printf("%s ", routerip);
                    strcpy(lastrouterip, routerip);
                }
                printf("%.3f ms ", rtt);
                if (icmp_hdr->icmp_type == ICMP_TIME_EXCEEDED) {
                } else if (icmp_hdr->icmp_type == ICMP_DEST_UNREACH) {
                    destreached = 1;
                }
            }
        }
        printf("\n");
    }
    close(sendsock);
    close(recv_sock);
    return 0;
}

```

Explanation :

For resolving the domains to IPs, I used the `gethostbyname()` function which ask the systems default DNS to resolve if its a domain name, so that the code works with both domain name or domain ip which is resolved by this function , which then returns a `hostent` containing the resolved address. I then needed two distinct sockets. First, I created `sendsock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)` to send off standard UDP probes. Second, I created `recv_sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP)` to catch the raw ICMP error messages sent back by routers (which is why this code requires `sudo` to run). I set up an outer for loop to increment my `ttl` (Time To Live) variable from 1 up to `MAXHOPS` (30). I used `setsockopt(sendsock, IPPROTO_IP, IP_TTL, &ttl, sizeof(ttl))` to forcefully restrict how far the packet could travel. To process the response, I implemented a `while (!check)` loop to call `recvfrom()` on my raw `recv_sock`. This is so because a raw socket captures all incoming ICMP traffic, not just the packets relevant to my trace. Within this loop, I capture the `end_time` immediately. If `recvfrom()` returns a value less than 0, the socket has reached its `RCVTIMEOUT` which is hardcode to 3 seconds can be changed via code and I break the loop to print an asterisk. . I incremented the destination port for every probe using `htons(DESTPORT + ttl + probe)` to match standard traceroute behavior, For the manual pointer arithmetic: I cast the buffer to an IP header using `struct ip *ip_hdr = (struct ip *)recv_buf`, calculated the header length with `ip_hdr->ip_hl << 2`, and skipped past it to find the ICMP payload using `struct icmp *icmp_hdr = (struct icmp *)(recv_buf + ip_hdr_len)`. the received packet contains a copy of the original IP header and the first 8 bytes of my original UDP packet. I skipped the 8-byte ICMP header using `struct ip *inner_ip = (struct ip *)((char *)icmp_hdr + 8)` and located the inner UDP header using `struct udphdr *inner_udp = (struct udphdr *)((char *)inner_ip + (inner_ip->ip_hl << 2))`. I then checked if `ntohs(inner_udp->uh_dport) == (DESTPORT + ttl + probe)`. Only if the destination port matched the unique port I sent I checked `flag = 1` to accept the packet. By checking `icmp_hdr->icmp_type`, I could determine if a router dropped the packet because the TTL expired (`ICMP_TIME_EXCEEDED`) or if the target host actually received it and

rejected the port (ICMP_DEST_UNREACH), at which point I flipped my destreached = 1 flag to successfully terminate the trace.

Output :

COMMAND USED :

```
gcc q3traceroute.c -o q3traceroute
sudo ./q3traceroute google.com
```

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_10/Part 2$ gcc q3traceroute.c -o q3traceroute
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_10/Part 2$ sudo ./q3traceroute google.com
traceroute to google.com (142.251.220.46), 64 hops max, 52 byte packets
 1  10.20.192.1  52.970 ms  55.408 ms  *
 2  10.200.10.14  63.190 ms  70.098 ms  68.125 ms
 3  103.147.138.250  48.285 ms  31.566 ms  59.282 ms
 4  117.232.137.122  66.077 ms  64.813 ms  47.343 ms
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  10.200.45.65  80.991 ms  * 65.142 ms
10  72.14.197.4  81.279 ms  75.873 ms  *
11  * * *
12  172.253.77.22  63.495 ms  72.14.237.138  79.197 ms  *
13  192.178.110.110  56.957 ms  72.14.236.219  64.658 ms  192.178.110.208  75.782 ms
14  * 142.250.226.135  84.442 ms  142.251.220.46  83.243 ms
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_10/Part 2$ sudo ./q3traceroute iitbhlai.ac.in
traceroute to iitbhlai.ac.in (192.168.10.115), 64 hops max, 52 byte packets
 1  10.20.192.1  39.170 ms  31.360 ms  39.978 ms
 2  10.200.10.14  31.963 ms  22.651 ms  18.474 ms
 3  192.168.10.115  21.749 ms  33.178 ms  50.048 ms
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/Desktop/CN/12340370_Ashish_10/Part 2$
```